

MATH2270/MATH2237/MATH2404 Assignment 3

Name

Mayank Anil Kantharia

Student number

S4083242

Assessment declaration checklist

Please carefully read the statements below and check each box if you agree with the declaration. If you do not check all boxes, your assignment will not be marked. If you make a false declaration on any of these points, you may be investigated for academic misconduct. Students found to have breached academic integrity may receive official warnings and/or serious academic penalties. Please read more about academic integrity [here](#). If you are unsure about any of these points or feel your assessment might breach academic integrity, please contact your course coordinator for support. It is important that you DO NOT submit any assessment until you can complete the declaration truthfully.

By checking the boxes below, I declare the following:

- ☒ I have not impersonated, or allowed myself to be impersonated by, any person for the purposes of this assessment
- ☒ This assessment is my original work and no part of it has been copied from any other source except where due acknowledgement is made.
- ☒ No part of this assessment has been written for me by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
- ☒ I have not used generative “AI” tools for the purposes of this assessment.
- ☒ Where this work is being submitted for individual assessment, I declare that it is my original work and that no part has been contributed by, produced by or in conjunction with another student.
- ☒ I give permission for my assessment response to be reproduced, communicated, compared and archived for the purposes of detecting plagiarism.
- ☒ I give permission for a copy of my assessment to be retained by the university for review and comparison, including review by external examiners.

I understand that:

- ☒ Plagiarism is the presentation of the work, idea or creation of another person or machine as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.
- ☒ Plagiarism includes the act of assisting or allowing another person to plagiarise or to copy my work.

I agree and acknowledge that:

- ☒ I have read and understood the Declaration and Statement of Authorship above.

- ☒ If I do not agree to the Declaration and Statement of Authorship in this context and all boxes are not checked, the assessment outcome is not valid for assessment purposes and will not be included in my final result for this course.

Assignment URL

Assignment URL: [Inflation & Food Expenditure Explorer](#)

References

- Roser, M., Ortiz-Ospina, E., & Ritchie, H. (2024). *Consumer price index* [Data set]. Our World in Data. Retrieved June 9, 2025, from <https://ourworldindata.org/grapher/consumer-price-index>
- Roser, M., Ortiz-Ospina, E., & Ritchie, H. (2024). *Share of consumer expenditure spent on food* [Data set]. Our World in Data. Retrieved June 9, 2025, from <https://ourworldindata.org/grapher/share-of-consumer-expenditure-spent-on-food>
- Chang, W., Cheng, J., Allaire, J. J., Xie, Y., & McPherson, J. (2023). *shiny: Web application framework for R* (Version 1.8.5) [Computer software]. CRAN. <https://CRAN.R-project.org/package=shiny>
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L., François, R., ... Yutani, H. (2023). *tidyverse: Easily install and load the tidyverse* (Version 2.0.0) [Computer software]. CRAN. <https://CRAN.R-project.org/package=tidyverse>
- Pebesma, E. (2023). *sf: Simple features for R* (Version 1.0-9) [Computer software]. CRAN. <https://CRAN.R-project.org/package=sf>
- South, A., Chamberlain, S., & Ramsay, J. (2023). *rnaturalearth: World vector map data from Natural Earth* (Version 0.3.0) [Computer software]. CRAN. <https://CRAN.R-project.org/package=rnaturalearth>
- South, A., Chamberlain, S., & Ramsay, J. (2023). *rnaturalearthdata: Download Natural Earth map data* (Version 0.1.0) [Computer software]. CRAN. <https://CRAN.R-project.org/package=rnaturalearthdata>
- Cheng, J., Karambelkar, B., & Xie, Y. (2023). *leaflet: Create interactive web maps with the JavaScript 'Leaflet' library* (Version 2.0.4) [Computer software]. CRAN. <https://CRAN.R-project.org/package=leaflet>
- Arel-Bundock, V. (2023). *countrycode: Convert country names and country codes* (Version 1.4.1) [Computer software]. CRAN. <https://CRAN.R-project.org/package=countrycode>
- Horner, J., O'Brien, E., & McPherson, J. (2023). *bslib: Bootstrap themes for shiny* (Version 0.5.1) [Computer software]. CRAN. <https://CRAN.R-project.org/package=bslib>
- Garnier, S. (2023). *viridis: Default color maps from 'matplotlib'* (Version 0.6.2) [Computer software]. CRAN. <https://CRAN.R-project.org/package=viridis>

Assignment Code

Preprocessing Code:

```
library(tidyverse)
library(sf)
library(rnaturalearth)
library(rnaturalearthdata)
library(countrycode)
library(rmapshaper)

cpi_raw <- read_csv("consumer-price-index.csv", show_col_types = FALSE)

df_inf <- cpi_raw %>%
  rename(
    Country = Entity,
    CPI = `Consumer price index (2010 = 100)`
  ) %>%
  group_by(Country) %>%
  arrange(Year) %>%
  mutate(
    Inflation = (CPI / lag(CPI) - 1) * 100
  ) %>%
  ungroup() %>%
  filter(!is.na(Inflation))

df_food <- read_csv("share-of-consumer-expenditure-spent-on-food.csv",
  show_col_types = FALSE) %>%
  rename(
    Country = Entity,
    FoodShare = `Food as share of total consumer expenditure`
  ) %>%
  select(Country, Year, FoodShare)

df <- inner_join(df_inf, df_food, by = c("Country", "Year")) %>%
  mutate(
    iso3 = countrycode(Country, "country.name", "iso3c")
  ) %>%
  filter(!is.na(iso3))
world_sf <- ne_countries(scale = "medium", returnclass = "sf") %>%
  select(iso_a3, name, geometry) %>%
  ms_simplify(keep = 0.05, keep_shapes = TRUE)

saveRDS(df, "data_prepped.rds")
message("data_prepped.rds written")

saveRDS(world_sf, "world_simp.rds")
message("world_simp.rds written")
```

UI and Server Code:

```
library(shiny)
library(tidyverse)
library(sf)
library(leaflet)
library(countrycode)
library(bslib)
```

```

library(viridis)

df    <- readRDS("data_prepped.rds")
world_sf <- readRDS("world_simp.rds")

pal_inf <- colorNumeric(viridis(256, option = "cividis"),
                        domain = df$Inflation, na.color = "transparent")
pal_food <- colorNumeric(viridis(256, option = "cividis"),
                        domain = df$FoodShare, na.color = "transparent")

ui <- navbarPage(
  title = "Global Inflation & Food Expenditure Explorer",
  windowTitle = "Inflation & Food Expenditure Explorer",
  theme = bslib::bs_theme(bootswatch = "flatly", version = 5),
  header = tags$head(
    tags$style(HTML("
      .navbar-brand { font-weight: bold; }
      .info-box { background-color: #f8f9fa; border-left: 4px solid #6c757d;
        padding: 10px; margin-bottom: 15px; }
      .metric-title { font-size: 1.2em; font-weight: bold; }
      .metric-value { font-size: 1.5em; font-weight: bold; color: #2c3e50; }
      .leaflet-container { border-radius: 5px; }
      .trend-stats { font-size: 0.85rem; }
      .trend-stats h5 { font-size: 1rem; margin-bottom: 0.25em; }
      .trend-stats .metric-title { font-size: 0.9rem; }
      .trend-stats .metric-value { font-size: 1.1rem; }
      .map-title { text-align: center; font-size: 1.3em; font-weight: bold;
        margin-bottom: 10px; }
    ")))
),

tabPanel(
  "Global Map",
  div(class = "plot-container",
    sidebarLayout(
      sidebarPanel(
        width = 4,
        h4("Map Controls", style = "margin-top: 0;"),
        radioButtons(
          "metric",
          tags$span(class = "metric-title", "Choose Metric:"),
          choices = c(
            "Food Spend Share" = "FoodShare",
            "Inflation" = "Inflation"
          ),
          selected = "FoodShare"
        ),
        sliderInput(
          "mapYear",
          "Year:",
          min = min(df$Year),
          max = max(df$Year),
          value = max(df$Year),
          sep = "",
        ),
        hr(),
        div(class = "about-box",

```

```

    h5("About this Map"),
    p("Explore global patterns of inflation and food expenditure. Toggle metric & year to explore
global patterns. Hover over countries to see exact values."),
    hr(),
    p(HTML(
      'Data Source: <a href="https://ourworldindata.org/grapher/consumer-price-index"
target="_blank">Consumer Price Index</a>,
      <a href="https://ourworldindata.org/grapher/share-of-consumer-expenditure-spent-on-food"
target="_blank">Food Spend Share</a>'
    ))
  ),
),
mainPanel(
  width = 8,
  tags$h2(
    "Mapping Inflation and Food-Budget Stress Globally",
    class = "map-title"
  ),
  leafletOutput("map", height = "550px")
)
)
),
),
tabPanel(
  "Country Trends",
  div(class = "plot-container",
    sidebarLayout(
      sidebarPanel(
        width = 4,
        h4("Trend Controls", style = "margin-top: 0;"),
        selectizeInput(
          "selCountries",
          "Select Countries:",
          choices = sort(unique(df$Country)),
          selected = c("United States", "India", "Australia"),
          multiple = TRUE,
          options = list(maxItems = 5)
        ),
        sliderInput(
          "yrRange",
          "Year Range:",
          min = min(df$Year),
          max = max(df$Year),
          value = c(min(df$Year), max(df$Year)),
          sep = ""
        ),
        hr(),
        uiOutput("trendStats"),
        hr()
      ),
      mainPanel(
        width = 8,
        plotOutput("trendPlot", height = "600px")
      )
    )
  )
)

```

```

),

tabPanel(
  "Country Profile",
  div(class = "plot-container",
    sidebarLayout(
      sidebarPanel(
        width = 4,
        h4("Country Selection", style = "margin-top: 0;"),
        selectizeInput(
          "compareCountry",
          "Choose Country:",
          choices = sort(unique(df$Country)),
          selected = "Australia",
        ),
        hr(),
        uiOutput("countryStats"),
      ),
      mainPanel(
        width = 8,
        plotOutput("comparePlot", height = "600px")
      )
    )
  )
),
),
)

server <- function(input, output, session) {
  year_data <- reactive({
    df %>% filter(Year == input$mapYear)
  })
  output$map <- renderLeaflet({
    leaflet(world_sf) %>%
      addProviderTiles("CartoDB.Voyager") %>%
      addPolygons(layerId = ~iso_a3,
        fillOpacity = 0.8, weight = 0.3, color = "white")
  })
  observe({
    mdf <- year_data()
    pal <- if (input$metric == "Inflation") pal_inf else pal_food

    map_df <- world_sf %>%
      left_join(mdf, by = c("iso_a3" = "iso3"))
    leafletProxy("map", data = map_df) %>%
      clearShapes() %>%
      addPolygons(fillColor = ~pal(get(input$metric)),
        fillOpacity = 0.8,
        weight = 0.3,
        color = "white",
        layerId = ~iso_a3,
        label = ~ifelse(
          is.na(get(input$metric)),
          paste0(name, ": No data"),
          sprintf("%s: %.1f%%", name, get(input$metric))
        ) %>%
      clearControls() %>%
      addLegend(position = "bottomright",

```

```

    pal    = pal,
    values = ~get(input$metric),
    title  = if (input$metric == "Inflation")
      paste0("Inflation (%) — ", input$mapYear)
    else
      paste0("Food share (%) — ", input$mapYear),
    labFormat = labelFormat(suffix = "%"),
    opacity  = 1)
  })

output$trendStats <- renderUI({
  req(input$selCountries, input$yrRange)

  stats_df <- df %>%
    filter(
      Country %in% input$selCountries,
      Year   >= input$yrRange[1],
      Year   <= input$yrRange[2]
    )
  if (nrow(stats_df) == 0) {
    return(div("No data in that range."))
  }
  avg_food <- mean(stats_df$FoodShare, na.rm = TRUE)
  min_food_row <- stats_df %>% slice_min(FoodShare, with_ties = FALSE)
  max_food_row <- stats_df %>% slice_max(FoodShare, with_ties = FALSE)
  avg_inf <- mean(stats_df$Inflation, na.rm = TRUE)
  min_inf_row <- stats_df %>% slice_min(Inflation, with_ties = FALSE)
  max_inf_row <- stats_df %>% slice_max(Inflation, with_ties = FALSE)
  div(class = "info-box trend-stats",
    h5("Food-Share Stats"),
    div(span(class="metric-title", "Average Food Share:"),
      span(class="metric-value", sprintf("%.2f%%", avg_food))),
    div(span(class="metric-title", "Min Food Share:"),
      span(class="metric-value", sprintf("%.2f%% (%s, %d)",
        min_food_row$FoodShare, min_food_row$Country, min_food_row$Year))),
    div(span(class="metric-title", "Max Food Share:"),
      span(class="metric-value", sprintf("%.2f%% (%s, %d)",
        max_food_row$FoodShare, max_food_row$Country,
max_food_row$Year))),
    hr(),
    h5("Inflation Stats"),
    div(span(class="metric-title", "Average Inflation:"),
      span(class="metric-value", sprintf("%.2f%%", avg_inf))),
    div(span(class="metric-title", "Min Inflation:"),
      span(class="metric-value", sprintf("%.2f%% (%s, %d)",
        min_inf_row$Inflation, min_inf_row$Country, min_inf_row$Year))),
    div(span(class="metric-title", "Max Inflation:"),
      span(class="metric-value", sprintf("%.2f%% (%s, %d)",
        max_inf_row$Inflation, max_inf_row$Country, max_inf_row$Year))),
  )
})

output$trendPlot <- renderPlot({
  req(input$selCountries)
  df %>%
    filter(
      Country %in% input$selCountries,

```

```

    Year >= input$yrRange[1],
    Year <= input$yrRange[2]
  ) %>%
  select(Country, Year, Inflation, FoodShare) %>%
  pivot_longer(
    cols = c(Inflation, FoodShare),
    names_to = "Metric",
    values_to = "Value"
  ) %>%
  ggplot(aes(x = Year, y = Value, color = Country)) +
  geom_line(size = 1.2) +
  facet_wrap(
    ~ Metric,
    scales = "free_y",
    ncol = 1,
    labeller = as_labeller(c(
      Inflation = "Annual Inflation (%)",
      FoodShare = "Food spend % of budget"
    ))
  ) +
  scale_color_viridis_d() +
  labs(
    title = "Inflation vs Food Budget Share for Selected Countries",
    x = NULL,
    color = ""
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "bottom",
    plot.title = element_text(face = "bold", size = 16),
    strip.text = element_text(face = "bold", size = 12)
  )
})

output$countryStats <- renderUI({
  req(input$compareCountry)
  country_data <- df %>%
    filter(Country == input$compareCountry)
  if(nrow(country_data) > 0) {
    stats <- country_data %>%
      summarise(
        latest_year = max(Year),
        latest_inflation = Inflation[Year == latest_year],
        latest_foodshare = FoodShare[Year == latest_year],
        avg_inflation_all = mean(Inflation, na.rm = TRUE),
        avg_foodshare_all = mean(FoodShare, na.rm = TRUE),
        min_year = min(Year),
        max_year = max(Year)
      )
    div(
      h4(input$compareCountry),
      div(style = "color: #666; font-size: 0.9em; margin-bottom: 15px;",
        paste("Data available from", stats$min_year, "to", stats$max_year)),
      hr(),
      h5("Latest Values:"),
      div(style = "margin-bottom: 10px;",
        span(class = "metric-title", paste("Inflation (", stats$latest_year, ")"),

```



```

      span(class = "metric-value", sprintf("%.1f%%", stats$latest_inflation))
    ),
    div(style = "margin-bottom: 15px;",
      span(class = "metric-title", paste("Food Share (", stats$latest_year, "): "),
      span(class = "metric-value", sprintf("%.1f%%", stats$latest_foodshare))
    ),
    hr(),
    h5("All Years Average:"),
    div(style = "margin-bottom: 10px;",
      span(class = "metric-title", "Average Inflation: "),
      span(class = "metric-value", sprintf("%.1f%%", stats$avg_inflation_all))
    ),
    div(style = "margin-bottom: 15px;",
      span(class = "metric-title", "Average Food Share: "),
      span(class = "metric-value", sprintf("%.1f%%", stats$avg_foodshare_all))
    ),
    hr(),
    div(style = "font-size: 0.8em; color: #999; margin-top: 10px;",
      paste("Based on", nrow(country_data), "years of data"))
  )
} else {
  div(p("No data available for this country."))
}
})

```

```

output$comparePlot <- renderPlot({
  req(input$compareCountry)
  df %>%
    filter(Country == input$compareCountry) %>%
    select(Year, Inflation, FoodShare) %>%
    pivot_longer(
      cols = c(Inflation, FoodShare),
      names_to = "Metric",
      values_to = "Value"
    ) %>%
    ggplot(aes(x = Year, y = Value, color = Metric)) +
    geom_line(size = 1.2) +
    facet_wrap(~ Metric, scales = "free_y", ncol = 1,
      labeller = as_labeller(c(
        Inflation = "Annual Inflation (%)",
        FoodShare = "Food spend % of total expenditure"
      ))) +
    scale_color_viridis_d(name = "Metric") +
    labs(
      title = paste("Inflation vs Food Budget Share for", input$compareCountry),
      x = NULL,
      color = ""
    ) +
    theme_minimal(base_size = 14) +
    theme(
      legend.position = "none",
      plot.title = element_text(face = "bold", size = 16),
      strip.text = element_text(face = "bold", size = 12)
    )
  })
}
shinyApp(ui, server)

```